

MongoDB



Forte concurrence dans le secteur des bases NoSQL

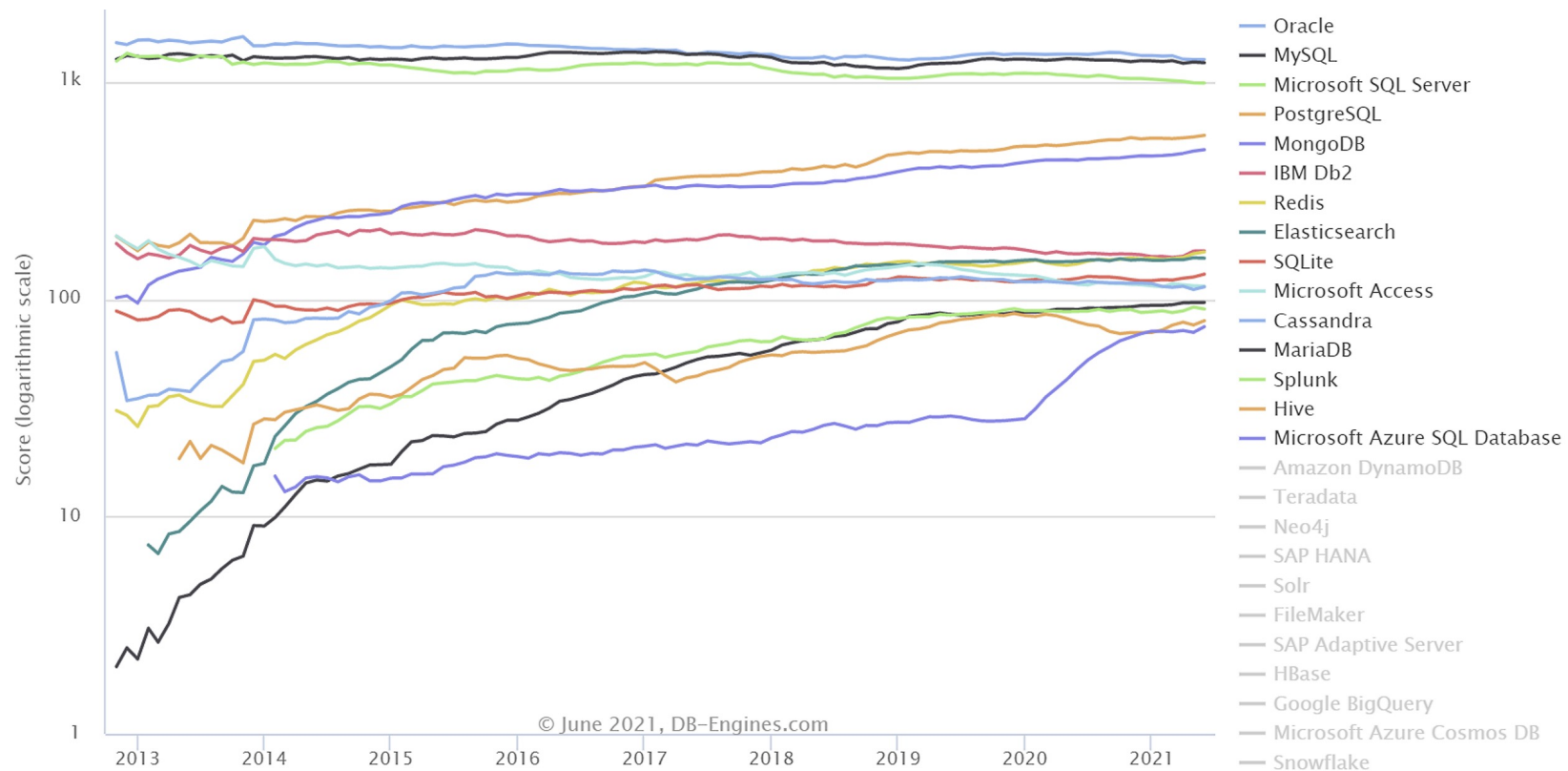
Comme toutes les sociétés basant leurs modèles sur « l'Opensource », MongoDB Inc. complète son offre avec des outils spécifiques (interface d'administration Compass par exemple) ou du support ... Payants

MongoDB Inc a une forte stratégie d'innovation :

- Beaucoup de versions se succèdent
- Le contour fonctionnel du produit évolue fortement (et intègre même des notions de graphes par exemple, ou de la BI, des éléments renforcés en termes de sécurité ainsi que des connexions vers des environnements Spark)
- Offre de services Cloud
- <https://www.mongodb.com/scale/mongodb-hosting-free>



Classement tendance (trend) des moteurs



Structure d'un document Json

Je suis un
document
JSON

```
{
  first_name: 'Paul',
  surname: 'Miller',
  cell: 447557505611,
  city: 'London',
  location: [45.123, 47.232],
  Profession: ['banking', 'finance', 'trader'],
  cars: [
    { model: 'Bentley',
      year: 1973,
      value: 100000, ... },
    { model: 'Rolls Royce',
      year: 1965,
      value: 330000, ... }
  ]
}
```

Fields

String

Number

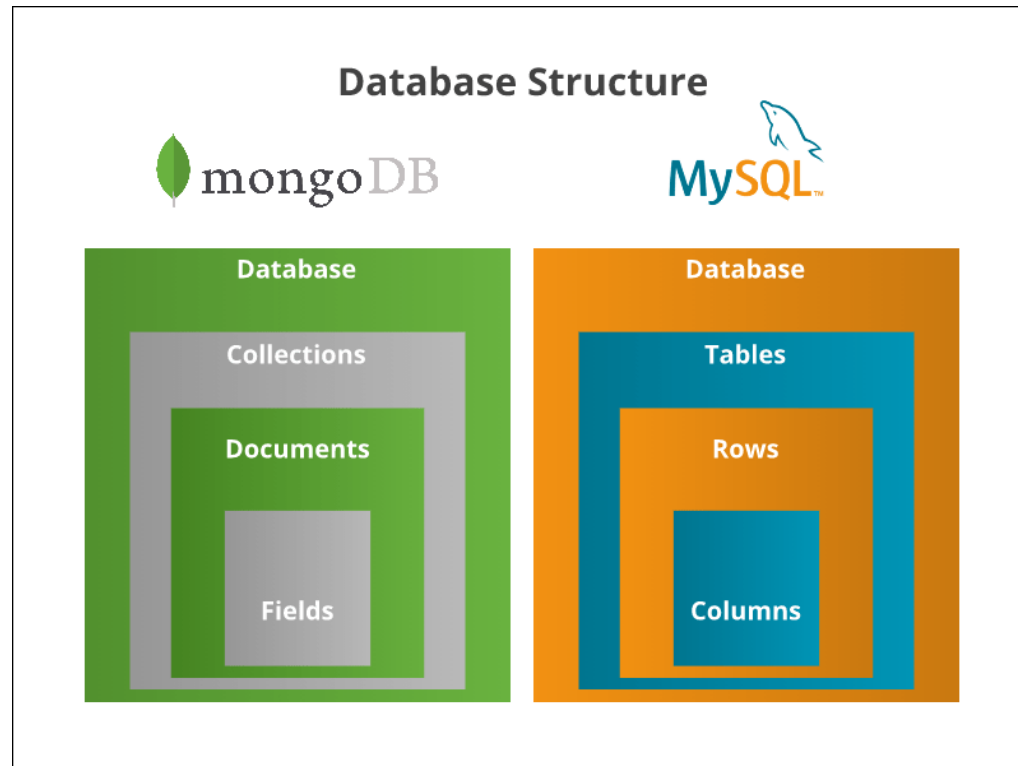
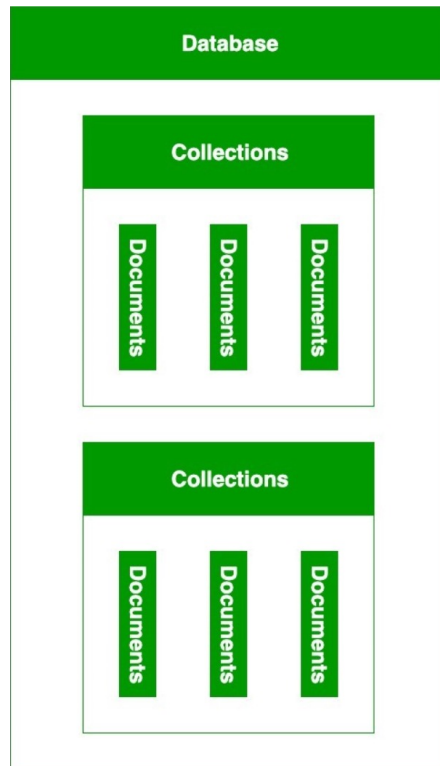
Geo-Coordinates

Typed field values

Fields can contain arrays

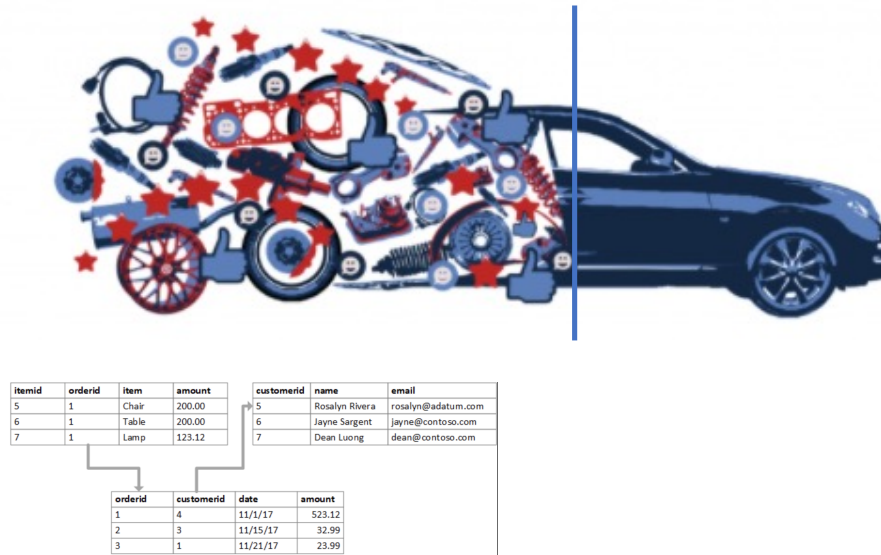
Fields can contain an array of sub-documents

MongoDB



Introduction au modèle orienté document

Modèle de données Relationnel



Représentation logique sous forme de tables à 2 dimensions

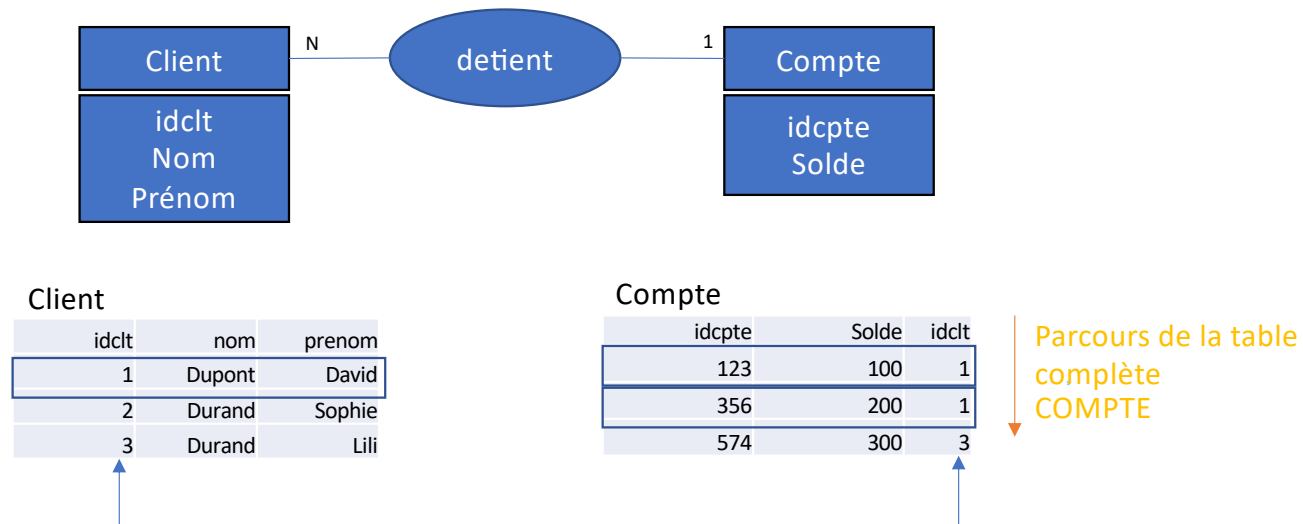
Modèle de donnée orienté Document



Représentation logique sous forme de documents

Introduction au modèle orienté document

L'approche relationnelle modélise les données de la manière suivante :



Comment obtenir le solde des comptes du Client David Dupont ?

```
Select idcpt, solde
from compte cpte left join client clt on clt.idclt = cpte.idclt
Where nom='Dupont' and prenom='David'
```

Le SGBDR a besoin de parcourir 2 tables pour collecter l'ensemble des données.

Introduction au modèle orienté document

Avec une approche document, les données nécessaires à l'application sont regroupées dans un et un seul document.

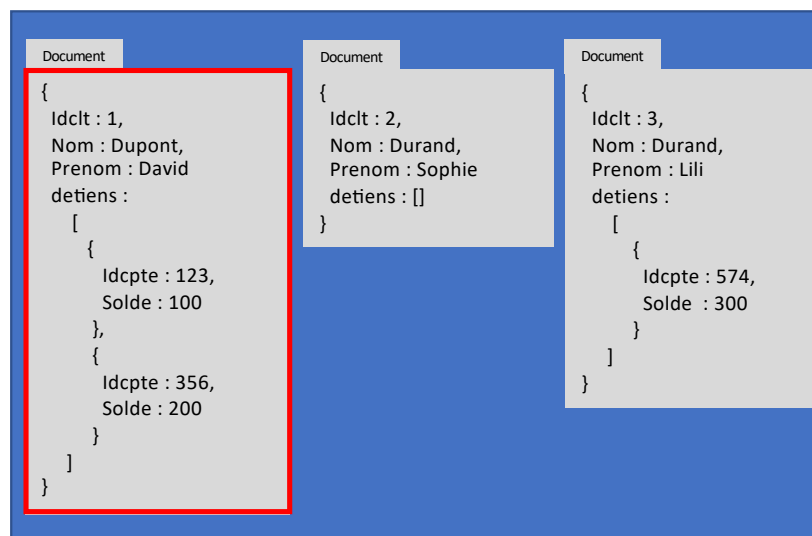
Client

idclt	nom	prenom
1	Dupont	David
2	Durand	Sophie
3	Durand	Lili

Compte

idcpt	Solde	idclt
123	100	1
356	200	1
574	300	3

Collection



- Plus besoin de jointures.
- Plus besoin de transactions si toutes les données à modifier sont réunies en un seul document.
- Facilement distribuable.
- Sous MongoDB, la taille d'un document ne peut excéder 16Mo.

Design Patterns

		Use Case Categories						
		Catalog	Content Management	Internet of Things	Mobile	Personalization	Real-Time Analytics	Single View
Patterns	Approximation	✓		✓	✓		✓	
	Attribute	✓	✓					✓
	Bucket			✓			✓	
	Computed	✓		✓	✓	✓	✓	✓
	Document Versioning	✓	✓			✓		✓
	Extended Reference	✓			✓		✓	
	Outlier			✓	✓	✓		
	Preallocated			✓			✓	
	Polymorphic	✓	✓		✓			✓
	Schema Versioning	✓	✓	✓	✓	✓	✓	✓
	Subset	✓	✓		✓	✓		
	Tree and Graph	✓	✓					

CRUD ?



CREATE

C



READ

R



UPDATE

U



DELETE

D



CRUD : Create

Comment créer un document dans une base mongo ?

```
db.products.insert( { item: "card", qty: 15 } )  
ou  
db.products.insertOne( { item: "card", qty: 15 } )
```



MongoDB collection **products**

```
{ "_id" : ObjectId("5063114bd386d8fadbd6b004"), "item" : "card", "qty" : 15 }
```

CRUD : Create

```
db.products.insert(  
  [  
    { _id: 11, item: "pencil", qty: 50, type: "no.2" },  
    { item: "pen", qty: 20 },  
    { item: "eraser", qty: 25 }  
  ]  
)
```



```
{ "_id" : 11, "item" : "pencil", "qty" : 50, "type" : "no.2" }  
{ "_id" : ObjectId("51e0373c6f35bd826f47e9a0"), "item" : "pen", "qty" : 20 }  
{ "_id" : ObjectId("51e0373c6f35bd826f47e9a1"), "item" : "eraser", "qty" : 25 }
```

Si une erreur survient lors de l'insertion d'un document, MongoDB continue d'insérer les documents suivants.

CRUD : Read

```
db.collection.find({condition},{projection}) return cursor
```

La condition est exprimée sous forme de document : { "champ" : valeur , ... }

Pour trouvez tous les restaurants dans le quartier (borough) de Brooklyn:

```
db.restaurants.find( { "borough" : "Brooklyn" } )
```

Maintenant, nous cherchons parmi ces restaurants ceux qui font de la cuisine italienne.

```
db.restaurants.find( { "borough" : "Brooklyn", "cuisine" : "Italian" } )
```


CRUD : Read

Extraire les champs voulus des documents filtrés

Un deuxième paramètre (optionnel) de la fonction find permet de choisir la ou les clés à retourner dans le résultat.

```
{ "champ" : 0|1 , ... }
```

Remarques :

- Pour les documents imbriqués, même approche objet que pour le filtre (':')
- La clé « _id », identifiant le document, est remontée systématiquement (=> { "_id" : 0 })
- true et false peuvent être utilisés à la place de 0 et 1.

```
db.restaurants.find( { "borough" : "Brooklyn", "cuisine" : "Italian" } ,{borough:0})
```

Affiche tout les champs de chaque document à l'exception du champs borough

```
db.restaurants.find( { "borough" : "Brooklyn", "cuisine" : "Italian" } ,{borough:1})
```

Affiche uniquement le champs borough de chaque document avec _id

CRUD : Read

`db.users.find({condition},{projection})` return cursor

```
{
  "_id" : 1,
  "name" : "sue",
  "age" : 19,
  "type" : 1,
  "status" : "P",
  "favorites" : { "artist" : "Picasso", "food" : "pizza" },
  "finished" : [ 17, 3 ]
  "badges" : [ "blue", "black" ],
  "points" : [ { "points" : 85, "bonus" : 20 }, { "points" : 85, "bonus" : 10 } ]
}
```



- Quelle requête retourne le document ci-dessous :
 - `db.users.find( { badges: "black" }, {name: 1} )`
 - `db.users.find( { "badges.0": "black" } )`
 - `db.users.find( { "favorites.artist": "Picasso" } )`
 - `db.users.find( { statu: "A", age: { $lt: 30 } } ).skip(0).limit(5)`



CRUD : Read

- Opérateurs de comparaisons : \$eq, \$gt, \$gte, \$in \$nin

```
db.inventory.find( { qty: { $nin: [ 5, 15 ] } } )
```

<https://docs.mongodb.com/manual/reference/operator/query-comparison/>

- Opérateurs logiques : \$and, \$or, \$not, \$nor

```
db.inventory.find( { price: { $not: { $gt: 1.99 } } } )
```

<https://docs.mongodb.com/manual/reference/operator/query-logical/>

- Opérateur sur les éléments : \$exists, \$type

```
db.inventory.find( { price: { $type : "double" } } )
```

<https://docs.mongodb.com/manual/reference/operator/query-element/>

- Opérateur de projection

```
Db. inventory.find( { points: { $elemMatch: {point: { $gt: 70 },bonus: { $gt:90 } } } },{ "grades.$": 1 } )
```

<https://www.mongodb.com/docs/manual/reference/operator/projection/>



CRUD : Read

```
db.addressBook.insertMany(
[
  { "_id" : 1, address : "2030 Martian Way", zipCode : "90698345","contact" : true },
  { "_id" : 2, address: "156 Lunar Place", zipCode : 43339374,"contact" : false },
  { "_id" : 3, address : "2324 Pluto Place", zipCode: NumberLong(3921412),"contact" : false },
  { "_id" : 4, address : "55 Saturn Ring" , zipCode : NumberInt(88602117) },
  { "_id" : 5, address : "104 Venus Drive", zipCode : ["834847278", "1893289032"],"contact" : true },
  { "_id" : 6, address : "222 Sun Boulevard", zipCode : null,"contact" : true }
]
)
```

```
db.addressBook.find(
  { $and : [
    {"zipCode" : { $type : "number" }},
    {"zipCode" : { $gt : 3000000}},
    {"zipCode" : { $lt : 50000000 } }
  ]
}
)
```

=> {2,3}

db.addressBook.find({ "zipCode" : { \$type : "string" } }) => {1,5}

db.addressBook.find({ "zipCode" : { \$not : { \$type : "null" } } }) => {1,2,3,4,5}

db.addressBook.find({ "contact" : { \$exists : true } }) => {1,2,3,5,6}

db.addressBook.find({ "contact" : { \$exists : true, \$eq : true } }) => {1,5,6}

CRUD : Read et les curseurs

CRUD : Read

- Un curseur (« cursor ») est un pointeur sur le résultat d'une requête.
- Une opération de lecture peut potentiellement retourner un ensemble de plusieurs centaines, milliers, ... de documents.
- Pour réduire l'impact mémoire et le trafic réseau MongoDB retourne un pointeur sur l'ensemble des documents satisfaisants la requête.
- Il est alors possible de réaliser des opérations sur cet objet.

Les méthodes « de base » :

- `limit()`: contraint la taille de l'ensemble résultat
- `count()`: modifie le curseur pour retourner le nombre de documents plutôt que les documents eux-mêmes.
- `skip()`: retourne un curseur qui pointe sur les documents résultat après avoir passé le nombre de documents indiqué.
- `pretty()`: configure le curseur pour afficher les documents de manière plus facile à lire (pour un humain)

Exemples :

- `db.trips.find({"usertype" : "Customer"}).count()`
- `db.zips.find({"state" : "CA"}).skip(500).limit(10)`

La liste complète :

<https://docs.mongodb.com/manual/reference/method/js-cursor/>



CRUD : Update

- Pour modifier les données, MongoDB fournit la commande update
 - `Db.mycollection.updateOne({$query},{set})` / modifie un seul document même si \$query retourne N documents
 - `Db.mycollection.updateMany({$query},{set})`
 - `db.collection.replaceOne({$query},{replacement})`
- MongoDB n'est pas transactionnel (ACID) :
 - Une commande qui met à jour plusieurs documents validera les modifications document par document.
 - Si une telle commande est interrompue, elle sera partiellement validée sur une partie des documents rendant éventuellement la base de données inconsistante.

```
db.restaurant.updateOne(  
  { "name" : "Central Perk Cafe" },  
  { $set: { "violations" : 3 } }  
);
```

```
db.restaurant.replaceOne(  
  { "name" : "Central Perk Cafe" },  
  { "name" : "Central Pork Cafe", "Borough" : "Manhattan" }  
);
```



Delete

`db.collection.deleteOne(filter,{options})`

Supprime le *premier* document correspondant au filtre **filter**. Si filtre {}, supprime le premier document retourné dans la collection. Utiliser un filtre sur '`_id`' pour être « précis ».



```
db.products.deleteOne( { $and : [{ item: "pencil" }, { type : "blue" }] } )
```

```
{ "acknowledged" : true, "deletedCount" : 1 }
```

`db.collection.deleteMany(filter,{options})`

Supprime *tous* les documents correspondants au filtre **filter**

Mongo_TP01_introduction.ipynb

Mongo_TP02_CRUD.ipynb



Aggregation framework

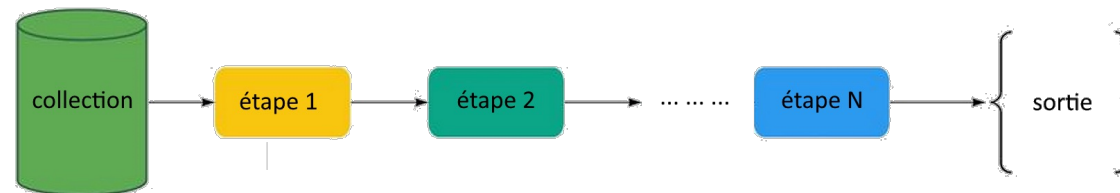


Aggregation framework

MongoDB intègre une plateforme qui permet d'enchaîner un ensemble d'opération l'une à la suite des autres.

Le pipeline d'agrégation est :

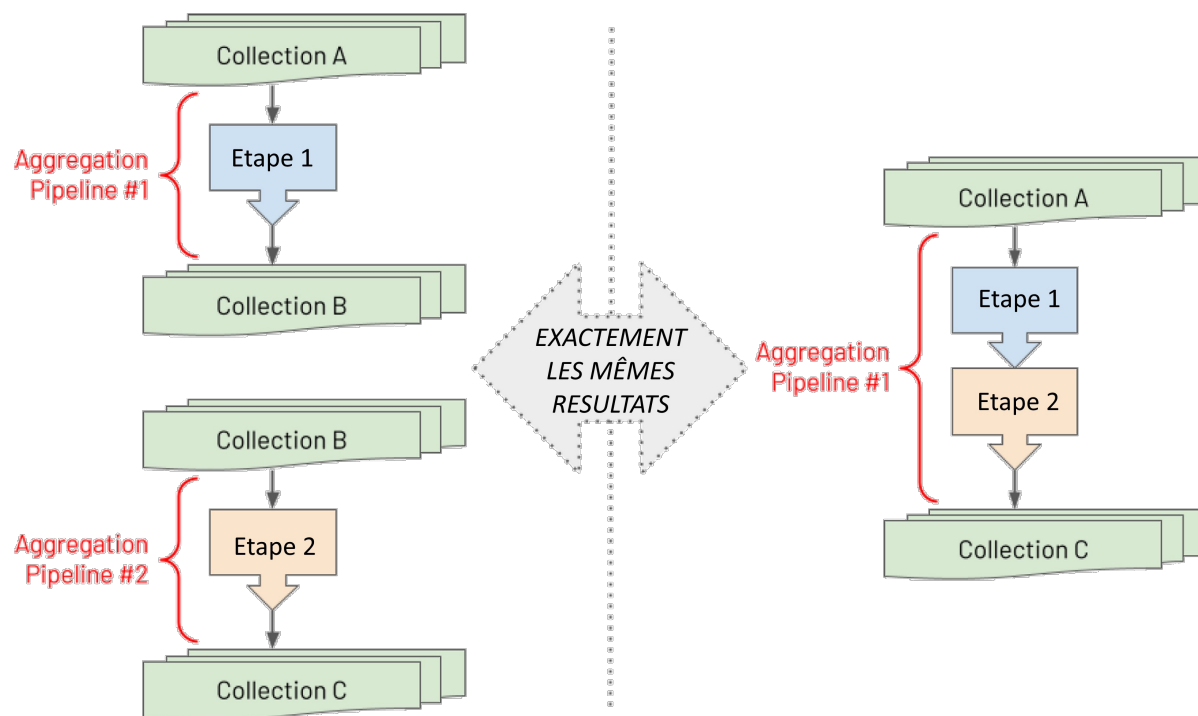
- - une série ordonnée d'opérations déclaratives appelées étapes
- - chaque étape réalise une transformation sur les documents en entrée
- - une étape peut exclure, synthétiser ou créer des documents
- - la totalité de la sortie d'une étape constitue l'entrée de l'étape suivante
- - la sortie finale est un curseur, une (nouvelle) collection ou une vue



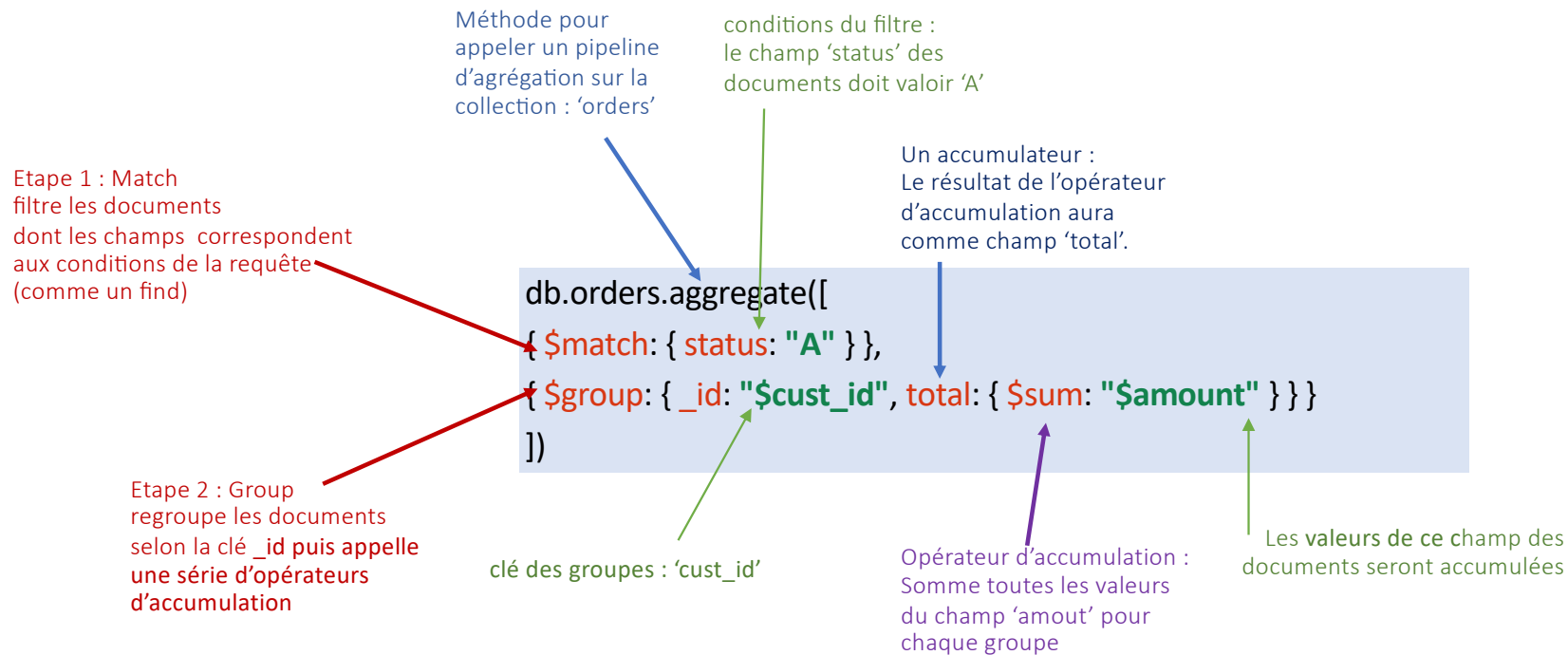
Aggregation framework

Les étages sont autonomes et sans état

Avec l'agrégation, vous pouvez prendre un problème complexe demandant une agrégation complexe et le décomposer en série d'étapes plus simples qui peuvent être testées et validées indépendamment.



Aggregation framework



Aggregation framework

Collection
↓
db.orders.aggregate([
 \$match stage → { \$match: { status: "A" } },
 \$group stage → { \$group: { _id: "\$cust_id", total: { \$sum: "\$amount" } } }
])

{ cust_id: "A123", amount: 500, status: "A" }
{ cust_id: "A123", amount: 250, status: "A" }
{ cust_id: "B212", amount: 200, status: "A" }
{ cust_id: "A123", amount: 300, status: "D" }

orders

\$match

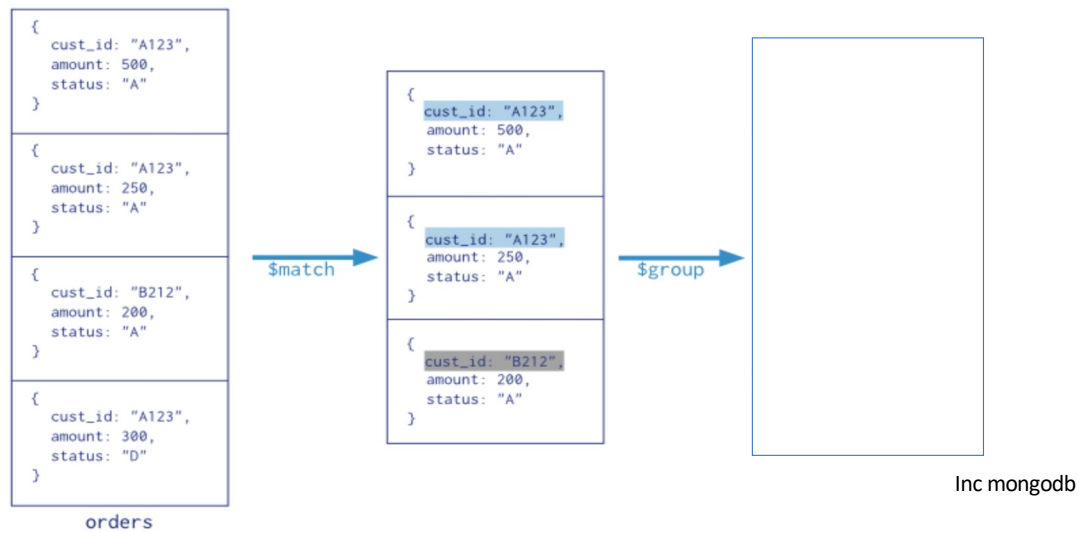
\$group

Inc mongodb



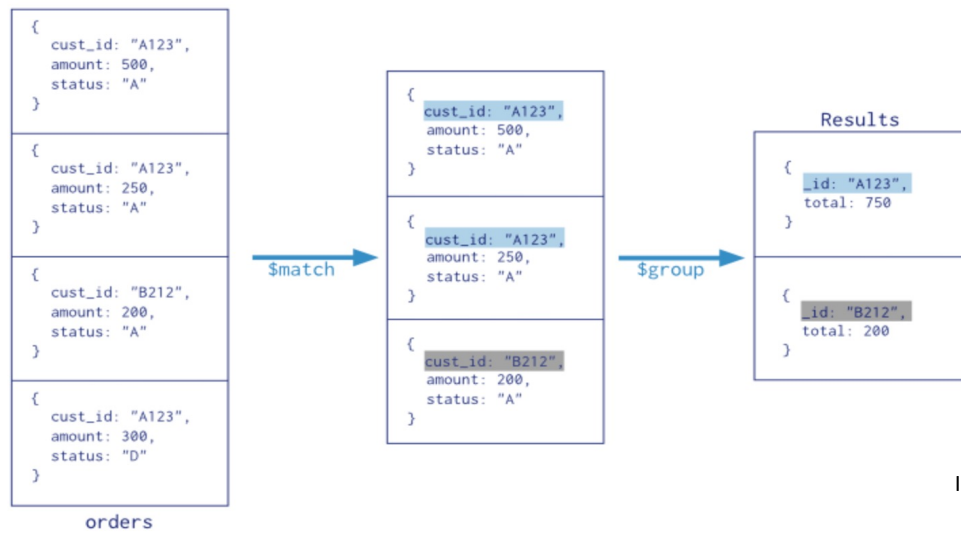
Aggregation framework

Collection
↓
db.orders.aggregate([
 \$match stage → { \$match: { status: "A" } },
 \$group stage → { \$group: { _id: "\$cust_id", total: { \$sum: "\$amount" } } }
])



Aggregation framework

Collection
↓
db.orders.aggregate([
 \$match stage → { \$match: { status: "A" } },
 \$group stage → { \$group: { _id: "\$cust_id", total: { \$sum: "\$amount" } } }
])



Inc mongodb

Aggregation framework



Les étapes peuvent servir à filtrer, projeter, trier, formater, ... les documents au fur et à mesure de leur avancement dans le pipeline.

La liste complète peut être retrouvée ici :

- <https://docs.mongodb.com/manual/reference/operator/aggregation-pipeline/>

Les étapes utilisent des opérateurs pour réaliser les opérations.

Ils sont au nombre de 135 et, comme les étapes peuvent être de différents types :

- Les opérateurs arithmétiques / Les opérateurs sur les tableaux / Les opérateurs booléens
- Les opérateurs sur les tailles / Les opérateurs sur les types / Les opérateurs sur les dates
- Les opérateurs sur les objets / Les opérateurs sur les chaînes / Les opérateurs trigonométriques

Mongo_TP02_aggregation_framework.ipynb

Mongo_TP03_aggregation_framework.ipynb

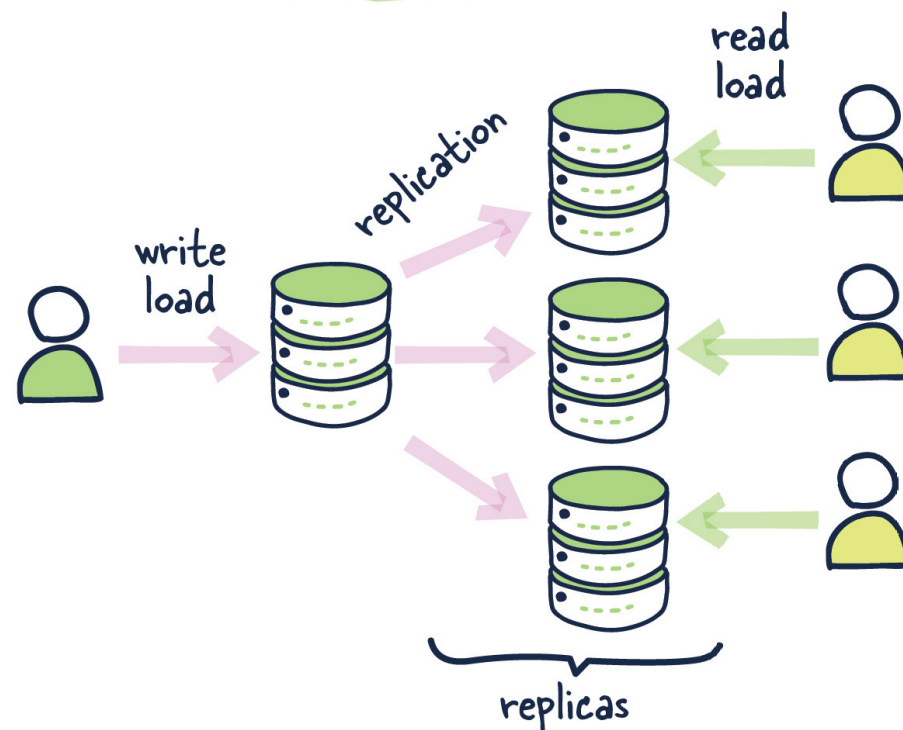


La réplication avec MongoDB

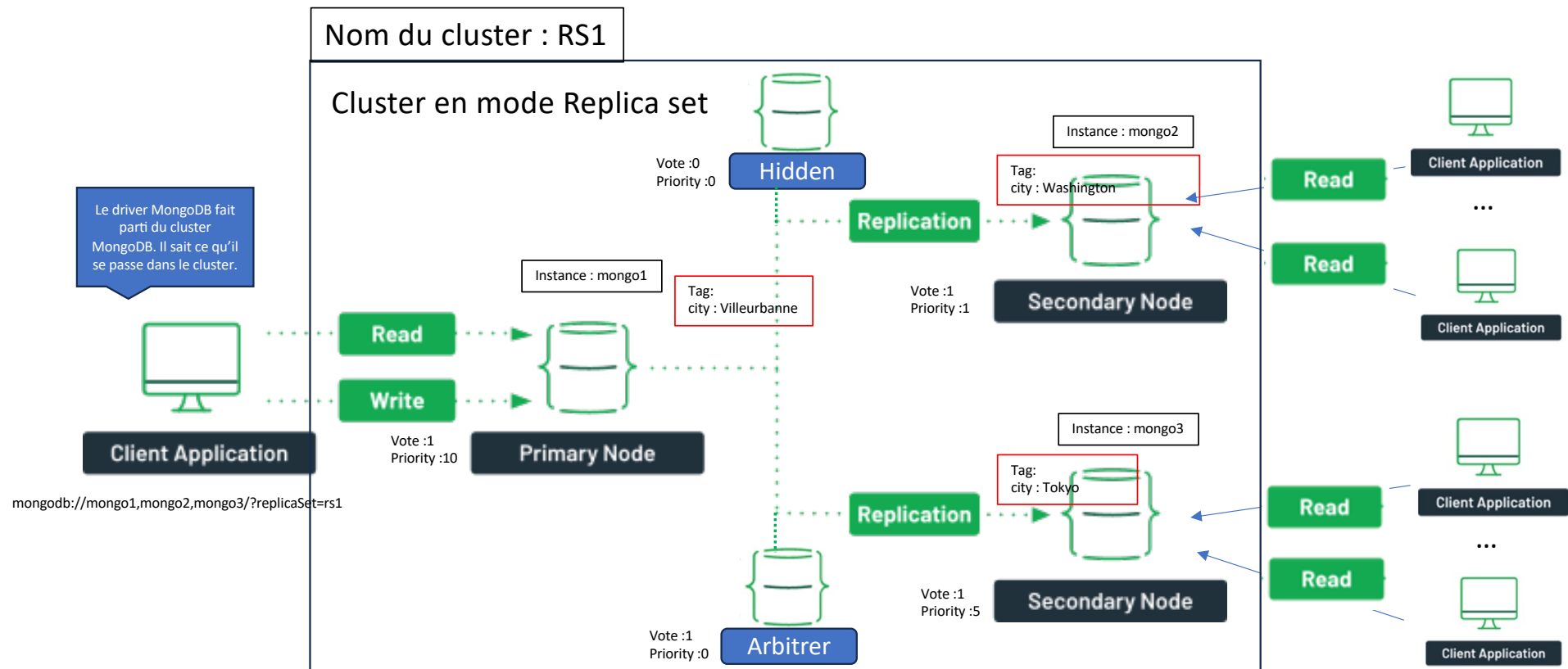


La réplication avec MongoDB

Scaling Reads



La réplication avec MongoDB



La réplication avec MongoDB

- ❑ Pour configurer un cluster en mode réplication, il sera nécessaire de déclarer les membres de votre cluster.

- ❑ Comment le primaire est élu ?
 - L'élection se fait avec un mécanisme de vote à la majorité(quorum)
 - Uniquement les membres ayant une carte de vote (vote:1) et une priorité (priority > 0) peuvent voter.
 - Pour définir des membres à privilégier comme primaire il suffit d'associer une priorité plus élevée que les autres membres.
 - Un membre avec une priorité de 0 ne peut pas devenir primaire mais il contiendra une copie des données.
 - Les membres avec un vote à 0 ne contribueront pas à l'élection du primaire.

- ❑ **Les rôles pour les membres :**
 - Primary : priority > 0 et vote =1
 - Secondary : priority > 0 et vote =1
 - Hidden: idem 'Secondary' mais non visible (vote=0 et priority=0). Il contient des données et peut être utilisé pour les backups.
 - Arbiter : utilisé lors du mécanisme d'élection du primaire (vote=1 et priority=0), il ne contient pas de données.

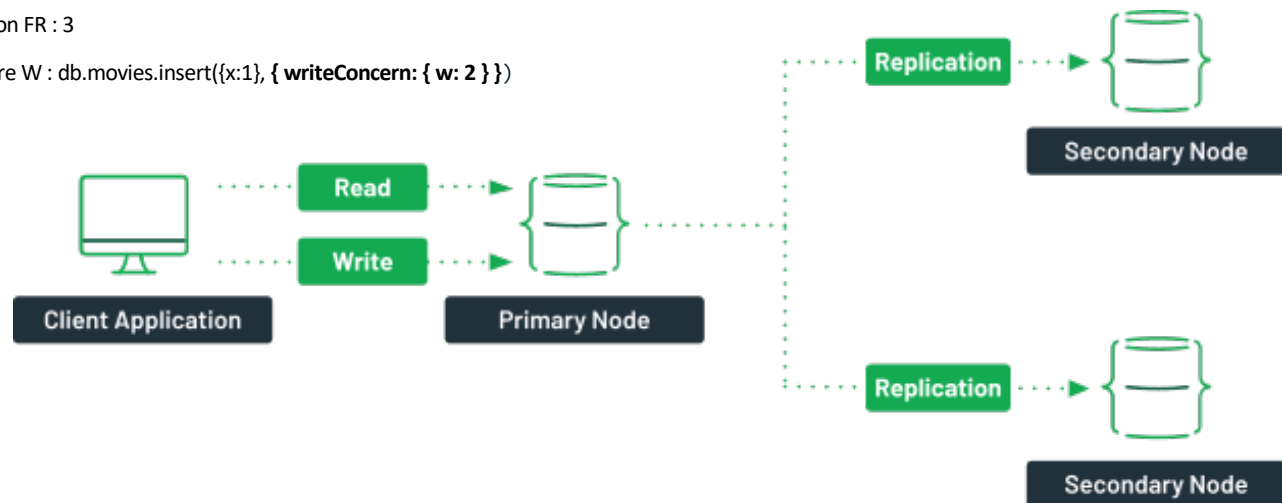
La réplication avec MongoDB

La cohérence des données lues est établie selon le niveau de cohérence demandé en écriture :

$$W + R > FR$$

Facteur de réplication FR : 3

Validation en écriture W : `db.movies.insert({x:1}, { writeConcern: { w: 2 } })`



Lecture cohérente
Validation en lecture R : 2
`db.movies.find().readConcern(majority)`



Lecture éventuellement incohérente
Validation en lecture R : 1
`db.movies.find().readConcern(local)`

Chap 6 : MongoDB_Replicaset_TP01

